

UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

Dipartimento di Ingegneria dell'Informazione
Corso di Laurea in Ingegneria Informatica

Relazione del corso di Algoritmi e Strutture Dati

Il Robot nel GridWorld:
Risoluzione mediante Programmazione Dinamica

Professoressa:
Marina Zanella

Componenti:
Singh Sahiljit (Matr. 740552)
Lublanis Matteo (Matr. 736418)

Anno Accademico 2025/2026

Indice

1 Compito 1 **2**
1.1 Descrizione del Problema . . . 2
1.1.1 Descrizione informale del problema . . . 2
1.1.2 Formalizzazione . . . 3
1.1.3 Ipotesi di raggiungibilità e terminazione . . . 4
1.2 Dimostrazione dell'applicabilità della Programmazione Dinamica . . . 6
1.2.1 Primo Segno Distintivo: Sottostruttura Ottima . . . 6
1.2.2 Secondo Segno Distintivo: Risoluzione dello stesso sottoproblema più volte . . . 6
1.3 Pseudocodice dell'Algoritmo . . . 8
1.3.1 Pseudocodice per il valore di una soluzione ottima . . . 8
1.3.2 Pseudocodice per la costruzione di una soluzione ottima . . . 10
1.4 Analisi Asintotica . . . 10
1.4.1 Complessità Temporale . . . 10
1.4.2 Complessità Spaziale . . . 14

2 Compito 2 **15**
2.1 Codifica dei due algoritmi . . . 15
2.2 Variante In-Place . . . 15
2.3 Vantaggi della Variante In-Place . . . 16

3 Compito 3 **17**
3.1 Istanze utilizzate per la sperimentazione . . . 17
3.2 Piattaforma utilizzata . . . 17
3.3 Risultati Sperimentali . . . 18
3.4 Verifica del numero di iterazioni K . . . 18
3.5 Confronto tra Analisi Asintotica e Risultati Sperimentali . . . 20
3.6 Grafici di Confronto . . . 21
3.7 Limitazioni riscontrate . . . 23
3.8 Conclusioni . . . 25

1 Compito 1

1.1 Descrizione del Problema

1.1.1 Descrizione informale del problema

È dato un ambiente discreto rappresentato da una griglia quadrata $N \times N$. Ogni cella è identificata dalla coppia di coordinate (r, c) con $0 \leq r, c \leq N - 1$ ed è di uno di due tipi:

- **Libera**: il robot può occuparla;
- **Ostacolo**: il robot non può occuparla.

Nella griglia sono individuate due celle libere particolari: la cella di partenza S (*Start*) e la cella obiettivo G (*Goal*).

Da ogni cella libera il robot può tentare uno spostamento in una delle quattro direzioni cardinali:

$$\mathcal{A} = \{\text{NORD},\ \text{SUD},\ \text{EST},\ \text{OVEST}\}.$$

Lo spostamento avviene se la cella di arrivo è interna alla griglia ed è libera, altrimenti il robot resta fermo nella cella corrente.

Evento	**Premio** R
Passo su una cella libera	-1
Urto contro un ostacolo o contro il bordo	-10
Raggiungimento della cella Goal	$+100$

Tabella 1: Struttura dei premi (funzione di ricompensa, vedere dopo) del modello.

Obiettivo: Determinare, per ogni cella libera della griglia, il valore della soluzione ottima V^* (ovvero il valore del cammino minimo tra tale cella e l'obiettivo) e ricostruire un cammino ottimo da S a G.

1.1.2 Formalizzazione

Il problema viene formalizzato come un Processo Decisionale di Markov (MDP) deterministico a tempo discreto definito dalla tupla $(\mathcal{S}, \mathcal{A}, \delta, R, \gamma)$, unitamente alle funzioni di valore (V^*, Q^*) e alla politica ottima (π^*):

- **Spazio degli stati** $\mathcal{S}$: L'insieme di tutte le celle libere della griglia $N \times N$:

$$\mathcal{S} = \{(r, c) : 0 \leq r, c \leq N-1,\ \text{Grid}[r, c] \neq \text{OSTACOLO}\}$$

  La cella obiettivo $G \in \mathcal{S}$ è definita come **stato terminale assorbente**. Si noti che in generale $|\mathcal{S}| \leq N^2$, con uguaglianza solo in assenza di ostacoli.

- **Spazio delle azioni** $\mathcal{A}$: L'insieme delle 4 direzioni cardinali ammissibili per il movimento del robot:

$$\mathcal{A} = \{\text{NORD},\ \text{SUD},\ \text{EST},\ \text{OVEST}\}$$

- **Funzione di transizione** $\delta : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$: Determina lo stato di destinazione in modo deterministico:

$$\delta(s, a) = \begin{cases} s' & \text{se la cella adiacente in direzione } a \text{ è libera e dentro la griglia} \\ s & \text{in caso di urto contro un ostacolo o il bordo} \end{cases}$$

- **Funzione di premio** $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$: Rappresenta la ricompensa immediata ottenuta eseguendo l'azione a dallo stato s:

$$R(s, a) = \begin{cases} c_{\text{urto}} = -10 & \text{se } \delta(s, a) = s \text{ (urto)} \\ c_{\text{raggiungimento}} = +100 & \text{se } \delta(s, a) = G \text{ (obiettivo raggiunto)} \\ c_{\text{passo}} = -1 & \text{altrimenti (spostamento valido)} \end{cases}$$

  Nota: Il costo $c_{\text{passo}} = -1$ incentiva il robot a trovare il cammino di lunghezza minima verso G.

- **Fattore di sconto** $\gamma = 1$: i premi futuri vengono valutati esattamente quanto quelli immediati, senza alcuna penalizzazione temporale. In combinazione con la struttura dei premi ciò garantisce che il cammino ottimo sia quello di lunghezza minima: percorrendo L mosse da S fino a G il ritorno complessivo vale $(s_0 \neq G)$

$$-(L-1) + 100 = 101 - L,$$

  quindi massimizzare il ritorno equivale esattamente a minimizzare L. Questa scelta ha però una conseguenza importante sulla terminazione dell'algoritmo, discussa nella sezione seguente.

Per risolvere l'MDP e determinare il cammino ottimo si definiscono tre entità fondamentali legate tra loro dalle Equazioni di Ottimalità di Bellman:

1. **Funzione Valore Azione-Stato Ottima** $Q^* : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$: Rappresenta il ritorno atteso eseguendo l'azione a nello stato s e proseguendo in modo ottimo da quel punto in poi:
$$Q^*(s, a) = R(s, a) + \gamma\, V^*(\delta(s, a))$$

2. **Funzione Valore Stato Ottima** $V^* : \mathcal{S} \to \mathbb{R}$: Rappresenta il massimo ritorno atteso scontato ottenibile a partire dallo stato s.

   - Poiché G è uno stato terminale assorbente da cui non si possono compiere ulteriori azioni, il suo valore è fisso:
   $$V^*(G) = 0$$

   - Per ogni altro stato libero $s \in \mathcal{S} \setminus \{G\}$, soddisfa l'**Equazione di Ottimalità di Bellman**:
   $$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a) = \max_{a \in \mathcal{A}} \{R(s, a) + \gamma\, V^*(\delta(s, a))\}$$

3. **Politica Ottima** $\pi^* : \mathcal{S} \setminus \{G\} \to \mathcal{A}$: È la funzione di decisione deterministica che associa a ogni stato l'azione ottima da compiere. Viene estratta a partire da $Q^*(s, a)$ mediante l'operatore arg max:
$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} Q^*(s, a) = \arg\max_{a \in \mathcal{A}} \{R(s, a) + \gamma\, V^*(\delta(s, a))\}$$
   Quando più azioni realizzano lo stesso valore massimo, l'arg max non è univoco: la politica ottima non è quindi necessariamente unica, mentre V^* lo è sempre.

1.1.3 Ipotesi di raggiungibilità e terminazione

Con $\gamma = 1$ la convergenza della Value Iteration non è garantita su qualunque istanza, ma richiede un'ipotesi esplicita sul dominio:

> **Ipotesi (R).** Da ogni cella libera $s \in \mathcal{S}$ esiste una sequenza di azioni che conduce alla cella obiettivo G.

L'ipotesi è necessaria: se una cella libera s non potesse raggiungere G, ogni azione disponibile in s produrrebbe un premio negativo (-1 oppure -10) senza mai incassare il premio terminale $+100$, e si avrebbe $V^*(s) = -\infty$. In quel caso il valore di s diminuirebbe

di almeno 1 a ogni iterazione, Δ non scenderebbe mai sotto ϵ e il ciclo `while` non terminerebbe.

Sotto l'ipotesi (R) la convergenza è invece garantita e, come si dimostrerà nell'analisi asintotica, avviene in un numero finito e limitato di iterazioni. si è dato ad entrambi gli pseudocodici un limite di sicurezza sul numero di iterazioni, che ne garantisce comunque la terminazione e segnala l'anomalia.

Come mai non $\gamma < 1$? Con un fattore di sconto $\gamma < 1$ la Value Iteration converge su qualunque istanza, anche senza l'ipotesi (R). Questa scelta è però inadatta al problema considerato. Un premio ottenuto dopo d mosse viene infatti pesato γ^{d-1} e nelle istanze a corridoio la distanza tra S e G è dell'ordine di N^2: con $\gamma = 0{,}95$, ad esempio, il premio del Goal raggiunto dopo 707 mosse (distanza tra S e G nell'istanza con $N = 40$) viene moltiplicato per $0{,}95^{706} \approx 2 \cdot 10^{-16}$. La Value Iteration si arresta allora prima che l'informazione sul Goal raggiunga le celle più lontane: la politica calcolata non conduce da S a G.

1.2 Dimostrazione dell'applicabilità della Programmazione Dinamica

1.2.1 Primo Segno Distintivo: Sottostruttura Ottima

Nel nostro problema:

- Sia $P^* = (s_0, s_1, s_2, \ldots, s_k = G)$ un cammino ottimo che conduce dalla cella di partenza $S = s_0$ alla cella obiettivo G.
- Se si considera una qualsiasi cella intermedia $s_i \in P^*$, il sottocammino $P^*_{i \to k} = (s_i, s_{i+1}, \ldots, G)$ rappresenta necessariamente la soluzione ottima al sottoproblema di raggiungere G a partire da s_i.
- Se infatti esistesse un sottocammino da s_i a G preferibile a $P^*_{i \to k}$, sostituendolo all'interno di P^* si otterrebbe un cammino globale da S a G migliore di P^*, il che contraddirebbe l'ottimalità di P^*.

Di conseguenza, la soluzione del problema principale per la cella s si compone direttamente a partire dalla soluzione ottima del sottoproblema relativo allo stato adiacente.

1.2.2 Secondo Segno Distintivo: Risoluzione dello stesso sottoproblema più volte

L'equazione di Bellman definisce V^* in modo circolare: $V^*(s)$ dipende dai valori delle celle adiacenti, che a loro volta dipendono da $V^*(s)$, poiché ogni mossa tra celle libere può essere compiuta anche in senso inverso; in caso di urto, inoltre, $\delta(s, a) = s$ e $V^*(s)$ dipende direttamente da se stesso. Una procedura ricorsiva costruita direttamente su questa equazione non terminerebbe mai.

Per ottenere una ricorrenza ben fondata si considera anche il numero di mosse ancora disponibili: sia $V_k(s)$ il massimo ritorno ottenibile partendo da S con al più k mosse. Vale

$$V_k(s) = \begin{cases} 0 & \text{se } k = 0 \text{ oppure } s = G \\ \max_{a \in \mathcal{A}} \{R(s,a) + \gamma\, V_{k-1}(\delta(s,a))\} & \text{se } k \geq 1 \text{ e } s \in \mathcal{S} \setminus \{G\} \end{cases}$$

Ogni chiamata ricorsiva riduce k di uno, quindi la ricorsione termina; come si dimostra nell'analisi asintotica, inoltre, V_k coincide con V^* non appena k raggiunge la distanza dal Goal della cella libera più lontana.

I sottoproblemi sono le coppie (s, k) e si sovrappongono: $V_{k-1}(s')$ serve al calcolo di V_k in tutte le celle che raggiungono s' con una mossa, cioè fino a quattro celle adiacenti, più s' stessa in caso di urto. Una ricorsione senza memorizzazione che calcoli $V_k(s)$ esegue

quindi un numero esponenziale di chiamate: ogni chiamata su una cella diversa da G ne genera quattro, una per azione, e al più una di queste raggiunge G e termina subito, per cui le chiamate sono almeno 3^k. I sottoproblemi distinti sono invece soltanto $|\mathcal{S}| \cdot (k+1)$. Nell'istanza con $N = 20$, ad esempio, la cella libera più lontana dista 123 mosse dal Goal: per calcolarne il valore ottimo servirebbero almeno $3^{123} \approx 5 \cdot 10^{58}$ chiamate, a fronte di $292 \cdot 124 = 36.208$ sottoproblemi distinti per l'intera istanza.

L'algoritmo `GRID-VALUE-ITERATION` risolve ciascun sottoproblema una sola volta, riempiendo la tabella dal basso per valori crescenti di k: l'iterazione k del ciclo `while` calcola V_k a partire da V_{k-1}, conservata in V_{old}. Poiché V_k dipende soltanto da V_{k-1}, è sufficiente conservare due livelli della tabella invece di tutti. Quando due livelli consecutivi coincidono ($\Delta = 0$) coincidono anche tutti i successivi, e la tabella contiene V^*.

1.3 Pseudocodice dell'Algoritmo

In questa sezione vengono presentati gli pseudocodici sviluppati secondo il paradigma della Programmazione Dinamica per la risoluzione del problema descritto. Il procedimento di calcolo si articola in due algoritmi distinti:

1. **Calcolo del valore della soluzione ottima:** l'algoritmo `GRID-VALUE-ITERATION` adotta uno schema iterativo *bottom-up* per calcolare la funzione valore ottima V^* su tutta la griglia. Durante il calcolo, aggiorna contemporaneamente una tabella ausiliaria π memorizzando le decisioni locali ottime.
2. **Costruzione della soluzione ottima:** l'algoritmo `CONSTRUCT-OPTIMAL-PATH` sfrutta le informazioni salvate nella tabella π dal primo algoritmo per tracciare e restituire il cammino ottimo sotto forma di sequenza ordinata di celle dalla partenza S all'obiettivo G.

1.3.1 Pseudocodice per il valore di una soluzione ottima

L'algoritmo `GRID-VALUE-ITERATION` riceve in ingresso la griglia, la cella obiettivo G, il fattore di sconto γ e la tolleranza $\epsilon > 0$. Il ciclo principale `while` esegue aggiornamenti successivi della tabella dei valori V fino a convergenza, ovvero fino a quando la massima variazione dei valori tra due iterazioni consecutive (Δ) risulta inferiore a ϵ. Il contatore k, confrontato con il limite $max_iter = N^2 + 1$, garantisce la terminazione anche quando l'ipotesi (R) non è soddisfatta.

Ingresso:

- $Grid[0..N-1, 0..N-1]$: matrice $N \times N$ rappresentante l'ambiente ($Grid[r, c]$ = LIBERA oppure OSTACOLO);
- N: dimensione del lato della griglia quadrata;
- $G = (r_G, c_G)$: coordinate della cella obiettivo (*Goal*);
- γ: fattore di sconto;
- ϵ: soglia di tolleranza per il criterio di convergenza ($\epsilon > 0$).

Uscite:

- $V[0..N-1, 0..N-1]$: tabella ausiliaria contenente i valori ottimi $V^*(s)$ per ogni stato $s = (r, c)$;

- $\pi[0..N-1, 0..N-1]$: tabella ausiliaria per la memorizzazione dell'azione ottima $\pi^*(s) \in \mathcal{A}$ da intraprendere da ciascuna cella $s = (r, c)$.

Per leggibilità il calcolo del massimo su $\mathcal{A}$ per una singola cella è isolato nella procedura ausiliaria `BELLMAN-UPDATE` (Algoritmo 1), che applica l'Equazione di Ottimalità di Bellman alla cella s leggendo i valori dell'iterazione precedente. Nel seguito, per $s = (r, c)$, si scrive $V[s]$ in luogo di $V[r, c]$ (analogamente per V_{old}, π e $Grid$).

Algoritmo 1 Aggiornamento di Bellman per una singola cella

BELLMAN-UPDATE(s, V_{old}, γ)
$max_q \leftarrow -\infty$; $best_a \leftarrow$ NIL
for $a \in \{\text{NORD}, \text{SUD}, \text{EST}, \text{OVEST}\}$
    do $s' \leftarrow \delta(s, a)$
    $q \leftarrow R(s, a) + \gamma \cdot V_{old}[s']$
    if $q > max_q$
        then $max_q \leftarrow q$; $best_a \leftarrow a$
return $(max_q, best_a)$

Algoritmo 2 Calcolo del valore di una soluzione ottima

GRID-VALUE-ITERATION$(Grid, N, G, \gamma, \epsilon)$
for $r \leftarrow 0$ **to** $N - 1$
    do for $c \leftarrow 0$ **to** $N - 1$
        do $V[r, c] \leftarrow 0$
        $\pi[r, c] \leftarrow$ NIL
$\Delta \leftarrow \infty$; $k \leftarrow 0$; $max_iter \leftarrow N^2 + 1$
while $\Delta \geq \epsilon$ **and** $k < max_iter$
    do $\Delta \leftarrow 0$; $k \leftarrow k + 1$
    $V_{old} \leftarrow V$
    for $r \leftarrow 0$ **to** $N - 1$
        do for $c \leftarrow 0$ **to** $N - 1$
            do $s \leftarrow (r, c)$
            if $Grid[s] \neq$ OSTACOLO **and** $s \neq G$
                then $(V[s], \pi[s]) \leftarrow$ BELLMAN-UPDATE(s, V_{old}, γ)
                $\Delta \leftarrow \max(\Delta, |V_{old}[s] - V[s]|)$
if $\Delta \geq \epsilon$
    then error "Nessuna convergenza: esistono celle libere da cui G è irraggiungibile"
return (V, π)

1.3.2 Pseudocodice per la costruzione di una soluzione ottima

Calcolata la tabella delle decisioni ottime π mediante `GRID-VALUE-ITERATION`, la ricostruzione della sequenza di celle da S a G non richiede ulteriori valutazioni della funzione di valore, ma un semplice attraversamento deterministico guidato dalla tabella π.
L'algoritmo `CONSTRUCT-OPTIMAL-PATH` compone la sequenza ordinata di coordinate $P = \langle s_0, s_1, \ldots, s_k \rangle$ con $s_0 = S$ e $s_k = G$. Per garantire la terminazione dell'algoritmo anche in presenza di griglie malformate o prive di soluzioni, viene introdotto un controllo sul numero massimo di passi eseguiti (N^2).

Algoritmo 3 Costruzione di una soluzione ottima

CONSTRUCT-OPTIMAL-PATH$(S, G, \pi, Grid, N)$
1: $s \leftarrow S$; $P \leftarrow$ Queue(); Enqueue(P, s); $steps \leftarrow 0$
2: **while** $s \neq G$ **and** $steps \leq N^2$
3: **do** $a^* \leftarrow \pi[s]$
4: **if** $a^* =$ NIL **or** $\delta(s, a^*) = s$
5: **then error** "Mossa ottima non definita o diretta contro un ostacolo/bordo"
6: $s \leftarrow \delta(s, a^*)$
7: Enqueue(P, s)
8: $steps \leftarrow steps + 1$
9: **if** $s \neq G$
10: **then error** "Goal non raggiungibile entro il numero massimo di passi consentiti"
11: **return** P

1.4 Analisi Asintotica

1.4.1 Complessità Temporale

Algoritmo `GRID-VALUE-ITERATION`**:**

- **Inizializzazione (righe 1–5):** I due cicli nidificati sulle righe r e colonne c scorrono tutte le $N \times N$ celle della matrice per azzerare V e impostare π a NIL. Ciascuna assegnazione richiede tempo $\Theta(1)$. La complessità dell'inizializzazione è dunque $\Theta(N^2)$.

- **Ciclo principale di convergenza (righe 6–14):** Sia K il numero totale di iterazioni del ciclo `while` eseguite prima che la variazione massima rispetti il criterio di arresto $\Delta < \epsilon$. In ciascuna delle K iterazioni vengono eseguite le seguenti operazioni:

  – La copia $V_{old} \leftarrow V$ (riga 8) duplica l'intera matrice $N \times N$ e costa quindi $\Theta(N^2)$;

– Scansione completa della griglia mediante due cicli `for` nidificati (righe 9–10) sulle $N \times N$ celle, per un totale di N^2 stati valutati;
– Per ogni cella ammissibile (non ostacolo e diversa da G), la chiamata a `BELLMAN-UPDATE` esegue il ciclo dell'Algoritmo 1 (righe 2–6), che valuta le $|\mathcal{A}| = 4$ direzioni cardinali. Il calcolo dello stato di destinazione $s' = \delta(s, a)$, la lettura di $R(s, a)$ e l'accesso in tabella a $V_{old}[s']$ avvengono in tempo costante $\Theta(1)$;
– L'aggiornamento dei valori max_q, $best_a$, $V[s]$, $\pi[s]$ e il calcolo del valore assoluto per Δ richiedono tempo $\Theta(1)$.

Il costo computazionale di una singola iterazione del ciclo `while` è quindi $\Theta(N^2) + \Theta(N^2 \cdot |\mathcal{A}|) = \Theta(N^2)$.

Moltiplicando il costo di una singola iterazione per il numero complessivo di iterazioni K, la complessità temporale di `GRID-VALUE-ITERATION` risulta:

$$T_{\text{Value-Iteration}}(N, K) = \Theta(N^2) + \Theta(K \cdot N^2) = \Theta(K \cdot N^2)$$

Limitazione del numero di iterazioni K. Il parametro K non è indipendente dall'istanza. Sia $d(s, G)$ la distanza minima da s a G, misurata in numero di mosse. Per $\gamma = 1$ e con inizializzazione $V_0 \equiv 0$ si dimostra per induzione che, dopo $k \geq 1$ iterazioni, per ogni cella $s \neq G$ vale

$$V_k(s) = \begin{cases} 101 - d(s, G) = V^*(s) & \text{se } d(s, G) \leq k \\ -k & \text{se } d(s, G) > k \end{cases}$$

Infatti $V_k(s)$ è il miglior ritorno ottenibile in al più k mosse: se G è raggiungibile entro k mosse conviene raggiungerlo per la via più breve, altrimenti la scelta migliore è compiere k passi su celle libere (premio -1 ciascuno), evitando gli urti (-10).

Ne segue che l'iterazione k modifica esattamente le celle con $d(s, G) \geq k$: quelle a distanza k passano da $-(k-1)$ a $101 - k$, quelle più lontane da $-(k-1)$ a $-k$. Detta $\text{ecc}(G) = \max_{s \in \mathcal{S}} d(s, G)$ l'**eccentricità** del Goal nel grafo delle celle libere (cioè quante mosse sono necessarie per raggiungere il Goal dalla cella libera più lontana), le iterazioni $1, \ldots, \text{ecc}(G)$ producono quindi $\Delta \geq 1$, mentre l'iterazione $\text{ecc}(G)+1$ non modifica alcun valore e produce $\Delta = 0$, arrestando il ciclo. Vale quindi l'uguaglianza

$$K_{\text{Standard}} = \text{ecc}(G) + 1$$

e, poiché un cammino minimo non ripete stati, $\text{ecc}(G) \leq |\mathcal{S}| - 1 \leq N^2 - 1$, da cui

$$K_{\text{Standard}} \leq |\mathcal{S}| \leq N^2.$$

Sostituendo nella formula precedente si ottiene la limitazione superiore per il caso peggiore:

$$T_{\text{Value-Iteration}}(N) = O(N^2 \cdot N^2) = O(N^4)$$

La limitazione vale anche quando l'ipotesi (R) non è soddisfatta, poiché in quel caso il ciclo viene interrotto dal limite di sicurezza dopo $max_iter = N^2 + 1$ iterazioni. Sotto l'ipotesi (R), invece, $K_{\text{Standard}} \leq N^2 < max_iter$ e il limite di sicurezza non interviene mai.

Il caso peggiore è raggiunto dalle istanze a corridoio unico (serpentine e labirinti a pettine), in cui l'unico cammino ammissibile attraversa una frazione costante delle celle libere e quindi $\text{ecc}(G) = \Theta(N^2)$. Nelle istanze poco ostruite si ha invece $\text{ecc}(G) = \Theta(N)$ e il costo scende a $\Theta(N^3)$. La verifica sperimentale di questa uguaglianza è riportata nel Compito 3.

Si osservi infine che, essendo i premi interi e $\gamma = 1$, i valori calcolati sono interi e la convergenza avviene in modo *esatto* (Δ diventa esattamente 0): la tolleranza ϵ non influenza quindi il numero di iterazioni, purché $\epsilon \leq 1$.

Algoritmo `CONSTRUCT-OPTIMAL-PATH`**:**

- **Inizializzazione (riga 1):** L'assegnazione dello stato iniziale, la creazione della coda P contenente S e l'azzeramento del contatore richiedono tempo $\Theta(1)$.
- **Ciclo di ricostruzione del cammino (righe 2–8):** Sia L la lunghezza del percorso estratto (in numero di passi), con $0 \leq L \leq N^2$. Il ciclo `while` viene eseguito una volta per ciascun passo del cammino ottimo, cioè L volte, e ogni iterazione aggiunge una cella a P:
  - Ad ogni passo, l'accesso alla tabella della politica $\pi[s]$ avviene in tempo $\Theta(1)$;
  - La funzione di transizione $\delta(s, a^*)$ e le operazioni di verifica sugli errori e sugli stalli richiedono tempo $\Theta(1)$;
  - L'operazione Enqueue(P, s) inserisce l'elemento in coda in tempo $\Theta(1)$.

  Il ciclo termina quando $s = G$ (oppure se viene raggiunto il limite di sicurezza di N^2 passi).

La complessità temporale di `CONSTRUCT-OPTIMAL-PATH` è quindi proporzionale alla lunghezza del cammino ottimo L:

$$T_{\text{Construct-Path}}(L) = \Theta(L + 1)$$

Nel caso peggiore in cui il cammino ottimo attraversi la quasi totalità delle celle libere della griglia allora $L = \Theta(N^2)$, mentre nel caso migliore $S = G$ e si ha $L = 0$. Di conseguenza possiamo affermare che la complessità temporale sia compresa, in generale, fra $\Omega(1)$ e $O(N^2)$.

Complessità Temporale Complessiva dell'intero procedimento:

$$T_{\text{Totale}}(N, K, L) = \Theta(K \cdot N^2) + \Theta(L + 1) = \Theta(K \cdot N^2) = O(N^4)$$

1.4.2 Complessità Spaziale

1. Memoria allocata da `GRID-VALUE-ITERATION`:

- **Matrice della griglia** $Grid[0..N-1, 0..N-1]$**:** occupa una quantità di memoria pari a $\Theta(N^2)$ celle;
- **Tabella dei valori** $V[0..N-1, 0..N-1]$**:** matrice di numeri reali di dimensione $N \times N$, pari a $\Theta(N^2)$ spazio;
- **Tabella della politica** $\pi[0..N-1, 0..N-1]$**:** matrice di elementi dell'alfabeto $\mathcal{A} \cup \{\text{NIL}\}$ di dimensione $N \times N$, pari a $\Theta(N^2)$ spazio;
- **Copia di lavoro** $V_{old}[0..N-1, 0..N-1]$ **(riga 8):** lo schema di aggiornamento **sincrono** richiede di conservare, per tutta la durata di ogni iterazione, l'intera tabella dei valori dell'iterazione precedente. Si tratta di una seconda matrice $N \times N$ di numeri reali, pari a ulteriori $\Theta(N^2)$ spazio. Questo è l'unico termine *ausiliario* in senso stretto, cioè non richiesto dalla specifica del problema né dal formato dell'uscita: è precisamente il termine che la variante in-place del Compito 2 elimina;
- **Variabili scalari ausiliarie:** le variabili s, Δ, k, max_iter e, in `BELLMAN-UPDATE`, $a, s', q, max_q, best_a$ occupano uno spazio costante $\Theta(1)$.

La complessità spaziale occupata dalle strutture tabulari è pertanto $\Theta(N^2)$, di cui $\Theta(N^2)$ di memoria *ausiliaria* imputabile alla sola copia V_{old}.

2. Memoria allocata da `CONSTRUCT-OPTIMAL-PATH`:

- **Struttura dati Coda** P**:** memorizza la sequenza delle coordinate delle celle del cammino ottimo da S a G. Nel caso di un cammino lungo L passi, la coda contiene $L+1$ coppie di coordinate (r, c), occupando uno spazio pari a $\Theta(L+1)$;
- **Variabili di controllo:** le variabili scalari $s, a^*, steps$ occupano memoria costante $\Theta(1)$.

Poiché il percorso P non può superare il numero totale di celle della griglia ($L \leq N^2$), lo spazio massimo richiesto dalla coda è $O(N^2)$.

Complessità Spaziale Complessiva dell'intero procedimento: Considerando tutte le tabelle ausiliarie (V, V_{old}, π) e la struttura dati della soluzione (P), la memoria complessiva richiesta è dominata dalle matrici $N \times N$:

$$S_{\text{Totale}}(N) = \Theta(N^2)$$

2 Compito 2

2.1 Codifica dei due algoritmi

L'implementazione dello pseudocodice è stata fatta in C++ nella seguente maniera:

- **Types.hpp**: Definisce i tipi di dati utilizzati (`CellType`, `Action`, `Position` e gli alias `Grid`, `ValueMatrix`, `PolicyMatrix`);
- **GridMDP.hpp**: Definizione delle firme dei metodi che verranno implementati in **GridMDP.cpp**;
- **GridMDP.cpp**: Implementazione dei metodi definiti in **GridMDP.hpp**. Le funzioni `gridValueIteration` e `constructOptimalPath` sono l'implementazione esatta dei due pseudocodici della scorsa sezione, mentre `gridValueIterationInPlace` realizza la variante descritta qui di seguito;
- **MemoryTracker.hpp / MemoryTracker.cpp**: Sostituiscono gli operatori globali `new` e `delete` per misurare il picco di memoria *heap* ausiliaria allocata da ciascuna variante, dato impiegato nella sperimentazione del Compito 3;
- **main.cpp**: Contiene il `main` e la logica per leggere i file di input, cronometrare le due varianti e salvare i risultati;
- **GenerateInstance.cpp**: Programma separato, usato una tantum per produrre le 10 istanze della cartella `istanze/`. Non fa parte dell'eseguibile di risoluzione.

La compilazione dell'eseguibile di risoluzione richiede i tre file `main.cpp`, `GridMDP.cpp` e `MemoryTracker.cpp`.

2.2 Variante In-Place

A differenza della versione standard (`gridValueIteration`), che calcola i nuovi valori basandosi su una copia congelata della matrice dell'iterazione precedente (`V_old`) secondo uno schema di aggiornamento sincrono, la variante in-place (`gridValueIterationInPlace`) aggiorna la funzione valore in modo asincrono:

- **Eliminazione della copia di matrice**: Non viene più allocata né duplicata la matrice $N \times N$ all'inizio di ogni iterazione del ciclo `while`.
- **Copia locale dello stato**: All'interno dei cicli di scansione, viene salvato solo il valore scalare della singola cella in esame (`v_old = V[r][c]`), necessario per calcolare la variazione assoluta $\Delta = |v_{old} - V(s)|$.

- **Aggiornamento in tempo reale**: Il nuovo valore di $V(s) = \max_a Q(s, a)$ viene sovrascritto immediatamente nella matrice condivisa V.

Di conseguenza, quando l'algoritmo valuta lo stato successivo s' per calcolare $q = R + \gamma \cdot V(s')$, legge i dati direttamente dalla matrice V in tempo reale. Se la cella adiacente s' è già stata elaborata durante la scansione corrente, l'algoritmo utilizzerà il suo valore già aggiornato al ciclo attuale anziché quello dell'iterazione del ciclo `while` precedente.

2.3 Vantaggi della Variante In-Place

Entrambe le varianti convergono alla medesima funzione valore ottimale V^*, che è unica. La politica π^* restituita può invece in linea di principio differire quando più azioni realizzano lo stesso valore massimo, poiché le due varianti visitano gli stati con valori intermedi diversi; nella sperimentazione del Compito 3 le due politiche sono comunque risultate identiche su tutte e 10 le istanze. Ciò premesso, l'approccio in-place offre due vantaggi:

1. **Maggiore velocità di convergenza**: Nella variante standard sincrona, il valore di una cella (es. la ricompensa del Goal) può propagarsi verso i vicini al massimo di una cella per ogni iterazione del ciclo `while`; come mostrato nel Compito 1 ciò porta a $K_{\text{Standard}} = \text{ecc}(G) + 1$ iterazioni esatte. Con l'aggiornamento in-place, si innesca un effetto a catena: se l'ordine di scansione incontra una cella adiacente a una già aggiornata, recepisce immediatamente la novità. Questo permette all'informazione di propagarsi lungo più celle nella stessa iterazione, riducendo il numero totale di cicli necessari a soddisfare la condizione di arresto $\Delta < \epsilon$. Il guadagno dipende però dall'orientamento dei cammini ottimi rispetto all'ordine di scansione per righe: se l'informazione deve propagarsi in verso opposto alla scansione, l'effetto a catena non si innesca e le due varianti compiono lo stesso numero di iterazioni. Si ha dunque $K_{\text{In-Place}} \leq K_{\text{Standard}}$, con uguaglianza nel caso sfavorevole.

2. **Efficienza di memoria**: Eliminando la duplicazione continua dell'intera griglia ad ogni iterazione, la complessità spaziale *ausiliaria* si riduce da $\Theta(N^2)$ a $\Theta(1)$, richiedendo soltanto la variabile temporanea v_{old}. Resta ovviamente invariata l'occupazione $\Theta(N^2)$ delle tabelle V e π, che costituiscono l'uscita dell'algoritmo e non sono quindi memoria ausiliaria.

Entrambi i fenomeni vengono misurati sperimentalmente nel Compito 3.

3 Compito 3

3.1 Istanze utilizzate per la sperimentazione

La valutazione delle prestazioni è stata condotta su un set di 10 istanze fisse caricate da file (`istanze/grid_1.txt` ... `istanze/grid_10.txt`) aventi dimensioni crescenti $N \in \{20, 25, 30, 40, 50, 60, 70, 80, 100, 120\}$, corrispondenti a griglie da 400 a 14.400 celle. Poiché lo spazio degli stati $\mathcal{S}$ comprende le sole celle libere, si ha $|\mathcal{S}| \leq N^2$: nelle istanze impiegate $|\mathcal{S}|$ varia da 292 a 9.798 (i valori esatti sono riportati nella Tabella 3).
Le topologie analizzate includono strutture non banali quali serpentine, labirinti a pettine, stanze multiple con colli di bottiglia, spirali concentriche e trappole a U; per visualizzarle basta aprire i relativi file di testo nella cartella `istanze/`. Tutte soddisfano l'ipotesi (R) del Compito 1: in ciascuna istanza ogni cella libera raggiunge il Goal, condizione necessaria alla convergenza con $\gamma = 1$.

3.2 Piattaforma utilizzata

Il benchmark è stato eseguito compilando i tre file `main.cpp`, `GridMDP.cpp` e `MemoryTracker.cpp` con ottimizzazioni attive:

```
g++ -std=c++17 -O2 -o gridmdp main.cpp GridMDP.cpp MemoryTracker.cpp
```

L'attivazione delle ottimizzazioni è essenziale: in build di *debug* il costo delle astrazioni della libreria standard domina il tempo di esecuzione e maschera l'andamento asintotico che si vuole misurare.

Caratteristiche della macchina di test:

- CPU: AMD Ryzen 7 5700X (8 core, 16 thread)
- RAM: 32 GB
- Sistema Operativo: Linux Mint 22.3
- Compilatore: g++ 13.3.0, standard C++17, ottimizzazione `-O2`

I tempi dipendono ovviamente dalla macchina; le iterazioni e l'occupazione di memoria sono invece deterministiche e riproducibili su qualunque piattaforma.

3.3 Risultati Sperimentali

I dati raccolti durante l'esecuzione del benchmark sono riassunti nella seguente tabella. Tabella, grafici e valori citati nel testo sono generati automaticamente a partire dal file `soluzioni/risultati_benchmark.txt` prodotto dall'eseguibile.

N	Variante	Celle (N^2)	Iterazioni	Tempo (ms)	Memoria Extra (KB)
20	Standard	400	124	0,328	3,59
	In-Place	400	73	0,159	0,00
25	Standard	625	49	0,254	5,47
	In-Place	625	49	0,202	0,00
30	Standard	900	82	0,437	7,73
	In-Place	900	31	0,138	0,00
40	Standard	1.600	745	6,551	13,44
	In-Place	1.600	374	2,524	0,00
50	Standard	2.500	159	3,251	20,70
	In-Place	2.500	95	1,494	0,00
60	Standard	3.600	119	3,480	29,53
	In-Place	3.600	119	2,894	0,00
70	Standard	4.900	139	4,036	39,92
	In-Place	4.900	139	3,398	0,00
80	Standard	6.400	159	7,166	51,88
	In-Place	6.400	159	6,070	0,00
100	Standard	10.000	199	15,742	80,47
	In-Place	10.000	199	13,778	0,00
120	Standard	14.400	4802	379,140	115,31
	In-Place	14.400	2462	188,524	0,00

Tabella 2: Iterazioni, tempo di esecuzione e memoria ausiliaria di picco per le due varianti.

La colonna **Memoria Extra** riporta il picco di memoria *heap* allocata durante la sola esecuzione dell'algoritmo, al netto delle tabelle V e π che costituiscono l'uscita e che vengono allocate dal chiamante. Misura quindi esattamente la memoria *ausiliaria* analizzata nel Compito 1.

3.4 Verifica del numero di iterazioni K

Il Compito 1 dimostra che per la variante sincrona vale $K_{\text{Standard}} = \text{ecc}(G) + 1$, dove $\text{ecc}(G)$ è l'eccentricità del Goal nel grafo delle celle libere. La tabella seguente confronta la previsione teorica con il valore misurato:

N	**Celle** (N^2)	$\lvert\mathcal{S}\rvert$	**ecc**(G)	**ecc**$(G)+1$ **(teorico)**	$K_{\textbf{Standard}}$ **(misurato)**
20	400	292	123	124	124
25	625	580	48	49	49
30	900	546	81	82	82
40	1.600	916	744	745	745
50	2.500	2.010	158	159	159
60	3.600	3.376	118	119	119
70	4.900	3.379	138	139	139
80	6.400	5.347	158	159	159
100	10.000	9.433	198	199	199
120	14.400	9.798	4.801	4.802	4.802

Tabella 3: Verifica dell'uguaglianza $K_{\text{Standard}} = \text{ecc}(G) + 1$.

Commento: la previsione è verificata **esattamente** su tutte e 10 le istanze, senza alcuno scarto. Si nota inoltre come nell'istanza $N = 120$ (serpentina) l'eccentricità vale 4.801 su 9.798 celle libere, cioè quasi metà dello spazio degli stati, e infatti K esplode a 4802 iterazioni contro le 199 dell'istanza $N = 100$, che pure ha una griglia comparabile. È questo, non la dimensione N, il fattore che determina il costo complessivo.

3.5 Confronto tra Analisi Asintotica e Risultati Sperimentali

Di seguito viene evidenziata la corrispondenza tra i valori teorici del Compito 1 e i dati misurati. Per non far dipendere le conclusioni dalla scelta arbitraria di due punti, gli andamenti sono stimati con una regressione ai minimi quadrati su scala logaritmica su tutti e 10 i punti: si assume il modello $y = k \cdot N^{\alpha}$ e si verifica che l'esponente stimato α sia prossimo a 2.

- **Complessità Spaziale**:
  - *Valore Teorico*: Standard = $\Theta(N^2)$, In-Place = $\Theta(1)$.
  - *Valore Misurato*: La versione Standard richiede la matrice duplicata V_{old} di dimensione $N \times N$ a valori `double` (8 byte per elemento), pari a $M_{\text{teorica}} = \frac{N^2 \times 8}{1024}$ KB:
    * Per $N = 20$: $M_{\text{misurata}} = 3{,}59$ KB (a fronte di 3,12 KB di soli dati).
    * Per $N = 120$: $M_{\text{misurata}} = 115{,}31$ KB (a fronte di 112,50 KB di soli dati).

    La regressione log-log sui 10 punti fornisce un esponente $\alpha = 1{,}94$, confermando l'andamento quadratico $\Theta(N^2)$.
    Per contro, la variante In-Place registra un picco di memoria ausiliaria **nullo** (0,00 KB per ogni valore di N): non effettuando alcuna allocazione dinamica oltre alle tabelle di uscita, verifica l'andamento $\Theta(1)$ nella forma più netta possibile.

- **Tempo di Esecuzione per una iterazione**:
  - *Valore Teorico*: $\Theta(N^2)$.
  - *Valore Misurato*: si considera il tempo medio di una singola iterazione, $t_{\text{iter}} = T/K$, che isola il costo di una scansione della griglia dal numero di scansioni necessarie. Ad esempio, per la variante Standard:
    * Per $N = 20$: $t_{\text{iter}} = \frac{0{,}328 \text{ ms}}{124} \approx 0{,}0026$ ms per iterazione.
    * Per $N = 120$: $t_{\text{iter}} = \frac{379{,}140 \text{ ms}}{4802} \approx 0{,}0790$ ms per iterazione.

    La regressione su tutti i punti fornisce $\alpha = 1{,}98$ per la variante Standard e $\alpha = 2{,}05$ per la In-Place: entrambi i valori sono compatibili con l'andamento teorico $\Theta(N^2)$.

- **Complessità Temporale Complessiva**:
  - *Valore Teorico*: $T(N, K) = \Theta(K \cdot N^2) = O(N^4)$, con $K_{\text{In-Place}} \leq K_{\text{Standard}}$.
  - *Valore Misurato*: La disuguaglianza $K_{\text{In-Place}} \leq K_{\text{Standard}}$ è verificata in tutte e 10 le istanze. Nell'istanza $N = 120$, l'aggiornamento In-Place riduce le iterazioni da 4802 a 2462 ($-48{,}7\%$) con un calo del tempo da 379,140 ms a 188,524 ms.

3.6 Grafici di Confronto

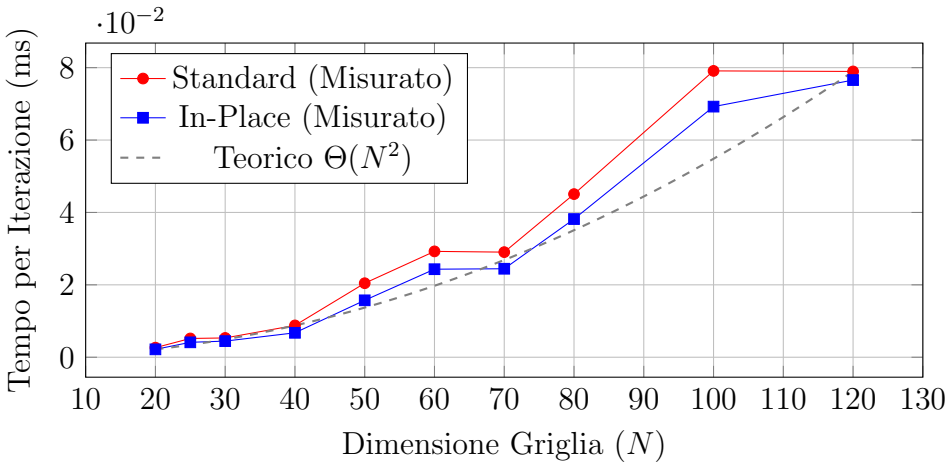


Figura 1: Confronto del tempo medio per una singola iterazione del ciclo `while`.

Commento: entrambe le varianti seguono la curva teorica quadratica $\Theta(N^2)$, confermando che il costo di una singola iterazione cresce quadraticamente con la dimensione della griglia.

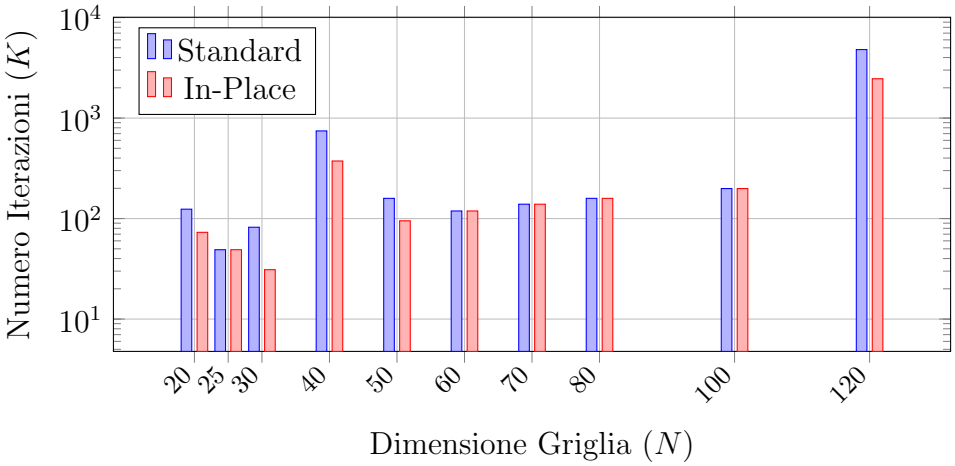


Figura 2: Confronto delle iterazioni di convergenza (K) tra Standard e In-Place. Scala logaritmica sulle ordinate.

Commento: si osserva come l'aggiornamento *In-Place* acceleri la convergenza nelle istanze con N pari a 20, 30, 40, 50 e 120, grazie alla propagazione immediata dei valori all'interno della stessa iterazione. Nelle istanze in cui i cicli si equivalgono (N pari a 25, 60, 70, 80 e 100) la geometria costringe la funzione valore a propagarsi in verso opposto alla scansione per righe dei cicli `for`, annullando l'effetto a catena. Si noti inoltre che K non cresce monotonamente con N: dipende dall'eccentricità del Goal, non dalla dimensione della griglia, come mostrato nella Tabella 3.

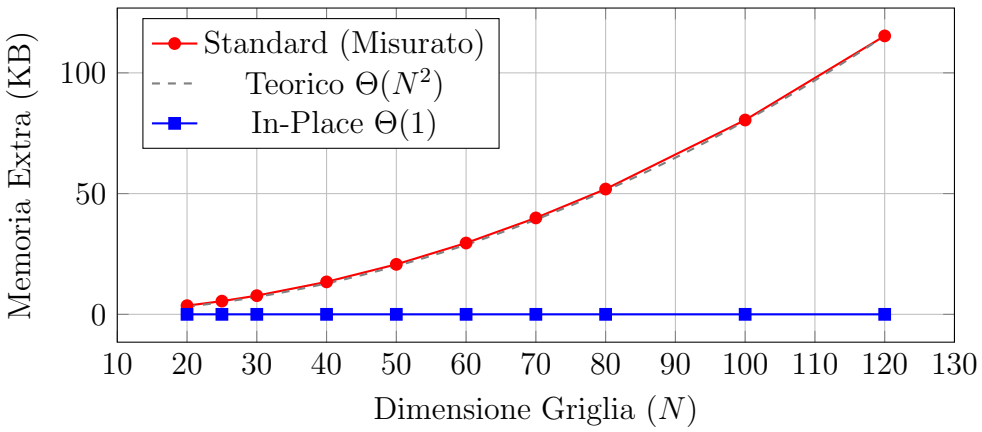


Figura 3: Verifica sperimentale della complessità spaziale ausiliaria.

Commento: la sovrapposizione tra stima teorica e misura empirica è pressoché perfetta. La serie In-Place resta costantemente nulla, a conferma che la variante non alloca alcuna memoria ausiliaria proporzionale a N.

3.7 Limitazioni riscontrate

Lo sviluppo e le prove di esecuzione hanno messo in evidenza le seguenti limitazioni:

- **Ipotesi di raggiungibilità**: con $\gamma = 1$ la convergenza richiede l'ipotesi (R) del Compito 1, soddisfatta da tutte le istanze di prova. Su un'istanza che la viola, ad esempio con una cella libera circondata da ostacoli su tutti i lati, entrambe le varianti si arrestano soltanto al limite di sicurezza di $N^2 + 1$ iterazioni e il programma segnala l'anomalia.

- **Fattore di sconto**: come discusso nel Compito 1, adottare $\gamma < 1$ per fare a meno dell'ipotesi (R) non è praticabile. In una prova con $\gamma = 0{,}95$, ottenuta modificando la costante `gamma` in `main.cpp`, il cammino ricostruito non raggiunge il Goal nelle istanze con $N = 40$ e $N = 120$, nelle quali la partenza dista dal Goal più di 700 mosse.

- **Dimensione delle istanze**: a limitare la dimensione delle istanze elaborabili è il tempo, non la memoria. Nelle istanze a corridoio K cresce come N^2 e il tempo come N^4, per cui raddoppiando N il tempo cresce di un fattore dell'ordine di 16: a fronte dei 379,140 ms misurati per la serpentina con $N = 120$, una serpentina con $N = 240$ richiede già alcuni secondi. La memoria cresce invece come N^2: anche con $N = 1000$ le tabelle occupano poche decine di MB. La variante In-Place annulla la memoria ausiliaria.

- **Vantaggio dell'In-Place non garantito**: la riduzione del numero di iterazioni dipende dall'orientamento dei cammini ottimi rispetto all'ordine di scansione per righe; nelle istanze con N pari a 25, 60, 70, 80 e 100 le due varianti compiono lo stesso numero di iterazioni.

3.8 Conclusioni

L'analisi dei dati empirici permette di trarre le seguenti conclusioni sulle prestazioni dei due algoritmi:

1. **Il numero di iterazioni è determinato dall'eccentricità del Goal**: la relazione $K_{\text{Standard}} = \text{ecc}(G) + 1$ è verificata esattamente su tutte e 10 le istanze. Ne consegue $K \leq |\mathcal{S}| \leq N^2$ e quindi il limite $O(N^4)$ per il caso peggiore, effettivamente avvicinato dalle istanze a corridoio unico ($N = 120$, 4802 iterazioni).

2. **Accelerazione della Convergenza**: Nelle istanze caratterizzate da percorsi orientati favorevolmente rispetto alla scansione (N pari a 20, 30, 40, 50 e 120) la variante **In-Place** riduce sensibilmente il numero di iterazioni: per $N = 120$ i cicli scendono da 4802 a 2462 ($-48{,}7\%$). L'aggiornamento immediato degli stati consente al valore di propagarsi lungo più celle adiacenti già all'interno della medesima iterazione.

3. **Impatto della Geometria degli Ostacoli**: Nelle istanze con N pari a 25, 60, 70, 80 e 100 entrambe le varianti richiedono lo stesso numero di cicli. Ciò accade quando i muri e i colli di bottiglia costringono la funzione valore a propagarsi in verso opposto rispetto a quello dei cicli `for`, rendendo inefficace l'effetto a catena dell'In-Place.

4. **Efficienza Temporale**: L'algoritmo In-Place risulta più rapido dello Standard su tutte le istanze misurate. Il vantaggio è marcato dove riduce le iterazioni (per $N = 120$ si passa da 379,140 ms a 188,524 ms) e resta comunque presente altrove, poiché anche a parità di iterazioni la variante evita la copia della matrice V_{old} a ogni ciclo.

5. **Complessità Temporale**: Viene confermata la complessità teorica $\Theta(K \cdot N^2)$: l'esponente stimato per il costo di una singola iterazione vale 1,98 per la Standard e 2,05 per la In-Place, entrambi compatibili con $\Theta(N^2)$.

6. **Complessità Spaziale**: Viene confermata la complessità spaziale teorica. La variante Standard scala quadraticamente ($\alpha = 1{,}94$) fino a richiedere 115,31 KB di memoria ausiliaria per $N = 120$, mentre la variante In-Place lavora direttamente sulla matrice principale senza allocare nulla, verificando l'andamento $\Theta(1)$.